

Write_up_BPE

Problem (unicode1):

(a):\x00

(b):The character's `__repr__()` shows an unambiguous/debug representation, while printing it shows how the character is actually displayed.

(c):In text, `chr(0)` is null, print it return space, when it is in the middle of string, just no effect and invisible

Problem(unnicode2):

(a):UTF-8 is commonly preferred because it is space-efficient for ASCII-heavy text and backward compatible with ASCII: English characters keep the same one-byte representation, while UTF-16 and UTF-32 usually use at least two or four bytes per character.

(b):“你好”.encode("utf-8").This function assumes that in UTF-8 encoding, one byte represents one character. However, in reality, two to three characters, or even four bytes, represent a Unicode character.

(c):b'\xe4\xbd'

Problem (train_bpe_tinystories):

(a):Training a byte-level BPE tokenizer with vocab size 10,000 on TinyStories using 16 workers / 64 chunks for parallel pre-tokenization took 1:53.59 wall-clock time and peaked at 201,092 KB (~196 MB) RSS. The longest token was b' accomplishment' (15 bytes), which makes sense because TinyStories is English prose and GPT-style pre-tokenization often learns frequent words together with their leading space.

(b):The py-spy flamegraph showed that most time was spent in the parallel pre-tokenization/count-construction phase, inside `count_chunk`. Within that phase, the main hotspots were `regex.finditer` over each chunk and converting each matched pre-token into a tuple of byte tokens before updating the Counter; the BPE merge loop itself was much smaller by comparison.

Problem (train_bpe_expts_owt):

(a):The longest OpenWebText token is a repeated mojibake pattern `ÃÂ...`, not a meaningful word, but it does make sense for byte-level BPE because the algorithm only merges frequent byte sequences and does not know whether the sequence is semantically meaningful.

(b):The TinyStories tokenizer mostly learns clean English story words, often with leading spaces, such as accomplishment and disappointment. In contrast, the OpenWebText tokenizer learns both long English words and web-specific artifacts such as repeated dashes, underscores, em dashes, and mojibake like ÃÃÃÂ..., which makes sense because OpenWebText is a much noisier web-crawl corpus.

Problem (tokenizer_experiments):

(a):Using 10 randomly sampled validation documents with random seed 42:

TinyStories tokenizer:

compression ratio = 4.03 bytes/token

OpenWebText tokenizer:

compression ratio = 4.25 bytes/token

(b):applying the TinyStories tokenizer to the sampled OpenWebText documents gives 3.17 bytes/token, much worse than the OpenWebText tokenizer's 4.25 bytes/token.

(c):My OpenWebText tokenizer processes about 0.82 MB/s when measured on the validation text with tokenizer loading and file I/O excluded. At this throughput, tokenizing the 825GB Pile would take roughly 278 hours, or about 11.6 days.

(d): uint16 can represent integer IDs from 0 to 65535, which is enough for both the 10K TinyStories vocabulary and the 32K OpenWebText vocabulary. It also stores each token in only 2 bytes, so it is more space-efficient than int32 or int64 while still covering the full tokenizer vocabulary.